

DataCross

Plataforma Inteligente para rotatividade de leitos

SPRINT 3 - ARQUITETURA LAMBDA E PIPELINE ORQUESTRADO

Turma: 1TSCPF

Equipe: Dados e Danados

Professor: Milton - MODERN DATA ARCHITECTURE & ENGINEERING

Integrantes:

Beatriz Falasca (RM 570086)

Felipe Ramos Lessa (RM 570626 - Representante)

Gabriel Rodrigues Costa (RM 568762)

Gabrielle Trindade Ferreira (RM 569092)

Maria Eduarda Rodrigues da Silva (RM 572951)



DATAACROSS

TECNOLOGIA QUE CONECTA SAÚDE

1. Sumário executivo e escopo da entrega

Este relatório documenta a implementação da Sprint 3 de Modern Data Architecture & Engineering para o projeto DataCross. A solução adota Arquitetura Lambda, com Batch Layer e Serving Layer implementadas no MVP e Speed Layer definida como evolução para integração FHIR/API em tempo real.

Resultado funcional comprovado

O pipeline Apache Airflow processou a base sintética de 10.000 internações, aplicou regras de qualidade, gerou 1.102 registros analíticos e realizou carga validada no Oracle Autonomous Database 26 AI, preservando 10.000 internações e 655 pacientes com status internado.

Exigência do professor	Atendimento no projeto	Evidência principal
Arquitetura Lambda/Kappa + justificativa	Lambda escolhida e comparada tecnicamente com Kappa	Seções 2 a 5
Diagrama da arquitetura	Diagrama Lambda com componentes implementados e evolução	Figura 1
DAG Apache Airflow	5 tarefas encadeadas: ingestão, transformação, validação, carga e pós-carga	Figuras 3 e Prints 06-07
Código da DAG	Código integral incorporado ao Anexo A deste relatório; trechos essenciais na Seção 12	Seção 12
Evidência de execução	Execução manual concluída com sucesso	PRINT 07
Logs e validação	Validação pré-carga, carga Oracle e reconciliação pós-carga	PRINTS 08-10
Camadas conceituais	Batch, Speed e Serving descritas por função, entrada, transformação e saída	Seções 4-5
PDF técnico	Participantes, justificativas, diagramas, pipeline, prints, logs e conclusão	Este arquivo

Observação de integridade: a base utilizada é sintética e serve para validação técnica do pipeline. Os indicadores apresentados não são tratados como estatísticas reais do SUS ou de hospitais reais.

2. Arquitetura escolhida: Lambda

A Arquitetura Lambda foi selecionada porque o DataCross precisa conciliar um processamento histórico em lote, já implementado no MVP, com a possibilidade de incorporar eventos hospitalares em fluxo em uma evolução futura. A escolha mantém continuidade com a arquitetura da Sprint 2, que priorizava carga batch e reservava FHIR/API em tempo real como evolução.

2.1 Comparação Lambda x Kappa

Critério	Lambda	Kappa	Decisão DataCross
Fonte atual	Adequada a arquivos e histórico em lote	Mais aderente a eventos contínuos	Lambda
MVP implementado	Batch nativo + serving analítico	Exigiria stream-first sem fonte real-time disponível	Lambda
Streaming futuro	Speed Layer pode ser adicionada sem abandonar o histórico	Seria o núcleo obrigatório da arquitetura	Lambda
Complexidade operacional	Maior, porém separa histórico e baixa latência	Menor em ambientes totalmente orientados a eventos	Aceitável para a evolução prevista
Aderência ao Challenge	Permite análises históricas e evolução operacional	Boa apenas se a principal fonte já for streaming	Lambda

Decisão técnica

Lambda é a opção mais coerente com o estágio atual: o MVP possui processamento batch funcional e consumo analítico no Oracle; a Speed Layer é explicitamente tratada como evolução, evitando afirmar que existe streaming implantado quando ele ainda não foi implementado.

3. Diagrama da Arquitetura Lambda proposta

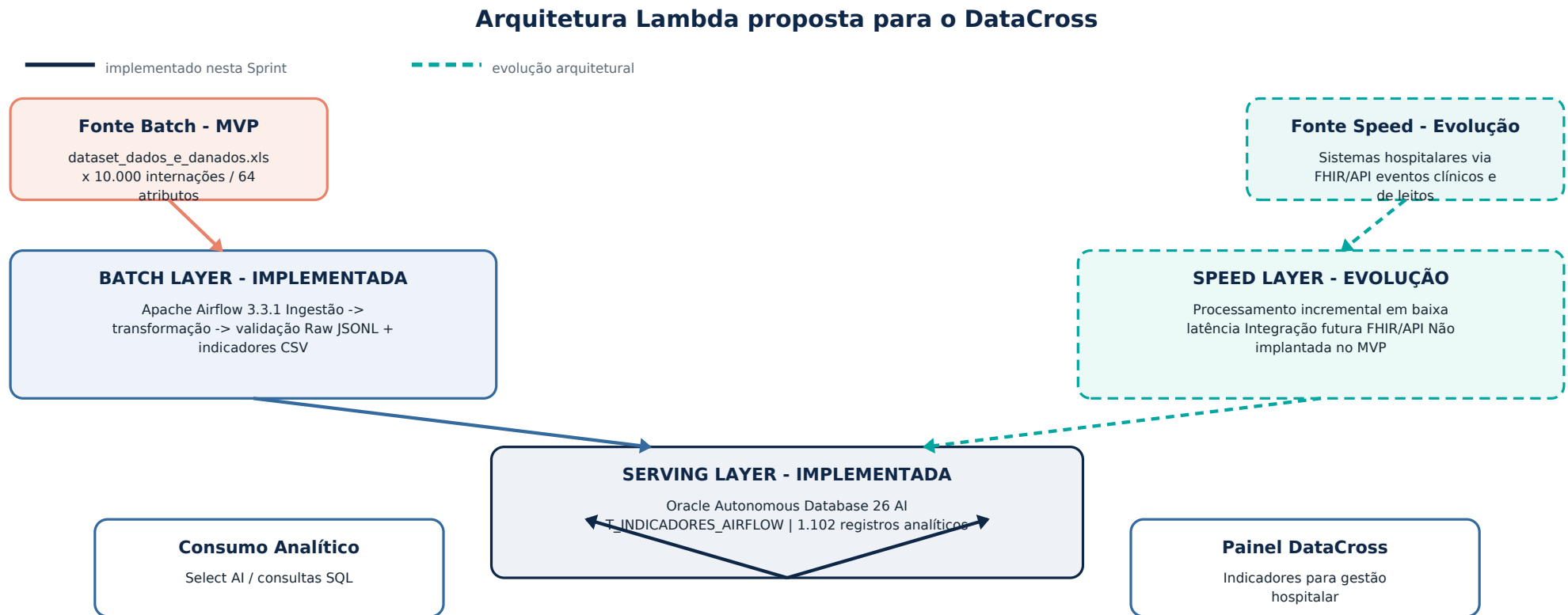


Figura 1 - Arquitetura Lambda do DataCross. Linhas sólidas representam componentes implementados nesta Sprint; linhas tracejadas representam evolução arquitetural planejada.

3.1 Leitura do diagrama

- Batch Layer: implementada com Apache Airflow 3.3.1, processando o arquivo dataset_dados_e_danados.xlsx.
- Raw/Processed: no laboratório, os artefatos intermediários são persistidos em volume Docker como JSONL e CSV; em produção, essa responsabilidade pode ser mapeada para Oracle Object Storage.
- Speed Layer: proposta para eventos FHIR/API de sistemas hospitalares; não implantada nesta Sprint.
- Serving Layer: implementada no Oracle Autonomous Database 26 AI por meio da tabela T_INDICADORES_AIRFLOW.
- Consumo: os dados podem alimentar consultas SQL/Select AI e o painel DataCross para apoio à gestão hospitalar.

4. Justificativa técnica segundo os critérios do enunciado

Critério	Justificativa aplicada ao DataCross
Volume dos dados	O MVP utiliza 10.000 internações e 64 atributos. A arquitetura foi desenhada para crescer para múltiplos hospitais, períodos e fontes sem alterar a separação entre ingestão, processamento e serving.
Processamento em lote	É indispensável no estágio atual, pois a fonte do laboratório é um dataset estruturado. A DAG implementa ingestão, transformação, validação e carga analítica de forma reproduzível.
Possibilidade de fluxo	A Speed Layer prevê eventos FHIR/API para movimentações e atualizações hospitalares de baixa latência. Ela é evolução arquitetural, não evidência de implementação atual.
Consumo analítico pelo painel	O pipeline produz uma tabela analítica por hospital, competência e diagnóstico, adequada a indicadores de internações, permanência, ocupação e custo estimado.
Integração com Oracle 26 AI	O worker do Airflow conecta-se por Wallet ao Oracle Autonomous Database e executa carga real na T_INDICADORES_AIRFLOW, seguida de reconciliação pós-carga.

Coerência com a Sprint anterior

A arquitetura mantém o Oracle Autonomous Database como repositório central e preserva a prioridade batch definida anteriormente. O fluxo real-time/FHIR permanece posicionado como evolução do DataCross.

5. Modelagem conceitual das camadas

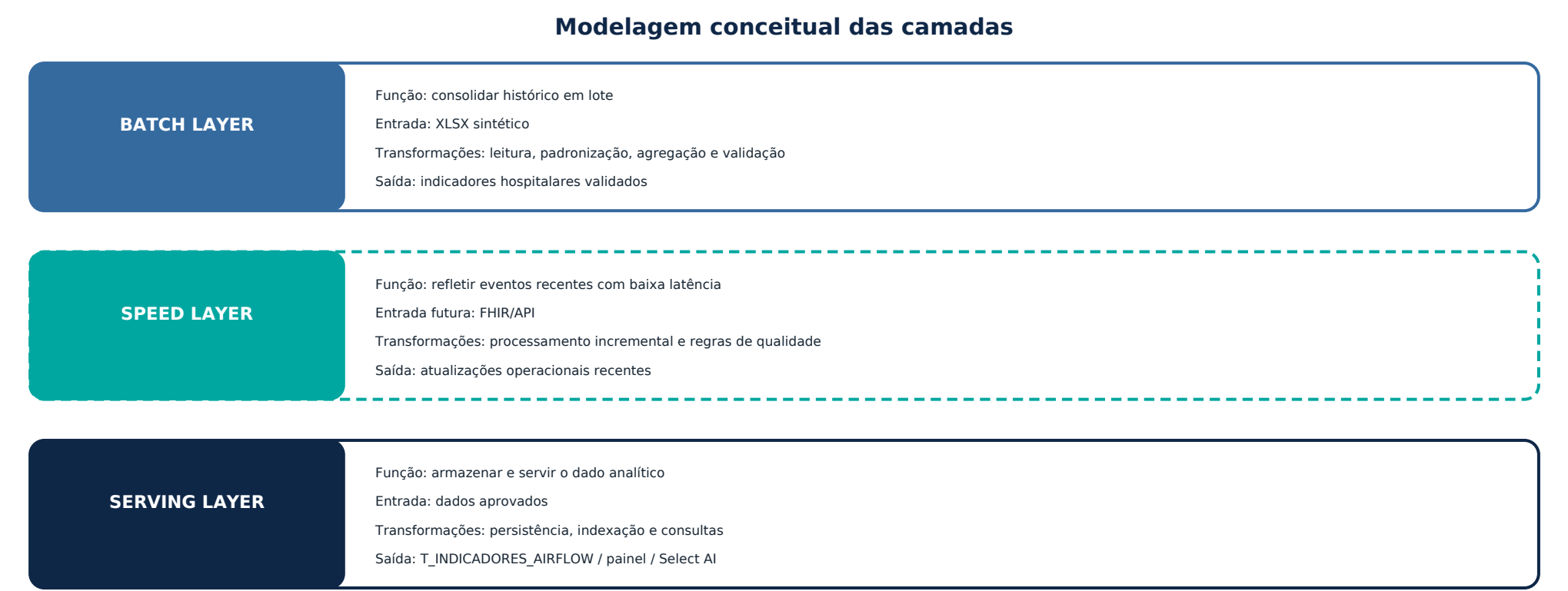


Figura 2 - Camadas conceituais da Arquitetura Lambda: função, entrada, transformação e saída de cada camada.

Camada	Função	Dados recebidos	Transformações	Dado disponibilizado
Batch	Consolidar dados históricos em lote	XLSX sintético com internações hospitalares	Leitura, padronização, conversão de tipos, agregação e qualidade	Indicadores consolidados por hospital + competência + diagnóstico
Speed (evolução)	Processar eventos recentes com baixa latência	Eventos futuros via FHIR/API	Tratamento incremental, validações e atualização de indicadores recentes	Eventos/indicadores recentes para consumo operacional
Serving	Centralizar o dado validado e disponibilizá-lo para análise	Saídas aprovadas das camadas de processamento	Persistência relacional, reconciliação e consultas analíticas	T_INDICADORES_AIRFLOW, SQL/Select AI e painel DataCross

6. Pipeline orquestrado com Apache Airflow

DAG datacross_pipeline - fluxo implementado

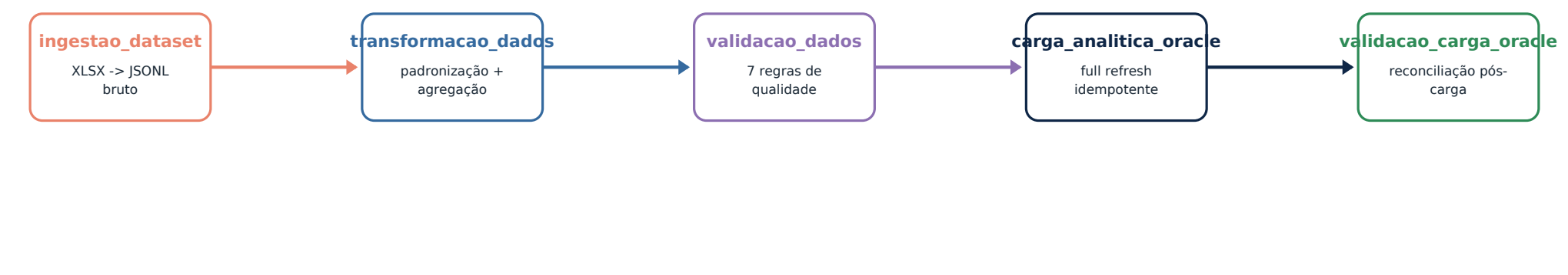


Figura 3 - Fluxo lógico da DAG datacross_pipeline implementada no Apache Airflow.

Tarefa	Responsabilidade	Saída / controle
ingestao_dataset	Lê o XLSX, verifica arquivo/colunas e grava cópia bruta JSON Lines	10.000 linhas recebidas + raw/prontuarios_raw.jsonl
transformacao_dados	Padroniza textos e datas, converte diária, calcula custo estimado e agrega	processed/indicadores_hospitalares.csv
validacao_dados	Executa sete regras de qualidade e bloqueia carga em caso de falha	validacao_pipeline.json + status APROVADO
carga_analitica_oracle	Cria/reutiliza T_INDICADORES_AIRFLOW e executa FULL REFRESH idempotente	1.102 registros analíticos no Oracle
validacao_carga_oracle	Consulta o Oracle e reconcilia registros/totais com a origem	10.000 internações, 655 internados, integridade OK

A DAG utiliza dependências explícitas entre tarefas. Uma etapa somente avança quando a anterior termina com sucesso, garantindo que dados reprovados na validação não sejam carregados na camada analítica.

7. Descrição detalhada das etapas do pipeline

7.1 Ingestão

A tarefa de ingestão lê dataset_dados_e_danados.xlsx no volume /opt/airflow/data. Antes de persistir o dado bruto, verifica a existência do arquivo, rejeita dataset vazio e confirma as colunas obrigatórias. A saída bruta é salva em JSON Lines para preservar uma zona de reproprocessamento.

7.2 Transformação

São padronizados status, hospital e diagnóstico; datas são convertidas; o valor da diária é convertido para número; e são construídos competência mensal, flag de internado e custo estimado. O resultado é agregado na granularidade hospital + competência + diagnóstico.

7.3 Validação

A qualidade é tratada como gate do pipeline. As regras cobrem volume preservado, chaves, coerência entre internados e total, permanência não negativa, taxa de ocupação entre 0 e 1 e ausência de duplicidade na granularidade analítica.

7.4 Carga analítica

A carga utiliza conexão mTLS com Wallet para o Oracle Autonomous Database 26 AI. A tabela T_INDICADORES_AIRFLOW é criada na primeira execução e reutilizada nas seguintes. A estratégia FULL REFRESH torna a DAG idempotente para o laboratório.

7.5 Validação pós-carga

Após o commit, a DAG consulta novamente o Oracle e compara registros analíticos, soma de internações e total de internados com os valores aprovados na origem. A execução somente é concluída após essa reconciliação.

Granularidade do data mart

Chave lógica: ID_HOSPITAL + COMPETENCIA + DOENCA_DETECTADA. Essa granularidade suporta comparações por unidade, mês e perfil clínico, sem duplicar as 10.000 internações de origem.

8. Evidências do ambiente e preparação

```
Administrator: Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and enhancements!
https://aka.ms/powershell

PS C:\windows\system32> docker --version
Docker version 29.4.1, build 055a478
PS C:\windows\system32> docker compose version
Docker Compose version v5.1.3
PS C:\windows\system32> |
```

PRINT 01 - Docker 29.4.1 e Docker Compose 5.1.3 instalados.

```
PS C:\DataCross_Airflow> "AIRFLOW_UID=500000" | Set-Content .env
PS C:\DataCross_Airflow> Get-Content .env
AIRFLOW_UID=500000
PS C:\DataCross_Airflow> Get-ChildItem -Force

Directory: C:\DataCross_Airflow

Mode                LastWriteTime         Length Name
----                -
d-----          8/28/2026   1:32 PM             config
d-----          8/28/2026   1:32 PM             dags
d-----          8/28/2026   1:32 PM             data
d-----          8/28/2026   1:32 PM             logs
d-----          8/28/2026   1:32 PM             plugins
d-----          8/28/2026   1:32 PM             wallet
-a-----          8/28/2026   1:32 PM              19 .env
-a-----          8/28/2026   1:32 PM          13039 docker-compose.yaml

PS C:\DataCross_Airflow> |
```

PRINT 02 - Estrutura do projeto DataCross_Airflow e arquivo .env.

O ambiente foi estruturado em Docker para isolar dependências do Apache Airflow e permitir reprodutibilidade. A pasta do projeto separa DAGs, dados, logs, plugins, configuração e Wallet.

```

PS C:\DataCross_Airflow> docker compose down
PS C:\DataCross_Airflow> docker compose up -d
[*] up 9/9
✓Network datacross_airflow_default          Created          9.8s
✓Container datacross_airflow-postgres-1      Healthy          32.4s
✓Container datacross_airflow-redis-1         Healthy          32.4s
✓Container datacross_airflow-airflow-init-1   Exited           32.3s
✓Container datacross_airflow-airflow-scheduler-1 Started          31.6s
✓Container datacross_airflow-airflow-apiserver-1 Healthy          52.2s
✓Container datacross_airflow-airflow-triggerer-1 Started          31.6s
✓Container datacross_airflow-airflow-dag-processor-1 Started          31.6s
✓Container datacross_airflow-airflow-worker-1 Started          52.3s
PS C:\DataCross_Airflow> docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED          STATUS
datacross_airflow-airflow-apiserver-1 datacross-airflow:3.3.1            "/usr/bin/dumb-init ..."          airflow-apiserver    About a minute ago    Up 29 seconds (healthy)
0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
datacross_airflow-airflow-dag-processor-1 datacross-airflow:3.3.1            "/usr/bin/dumb-init ..."          airflow-dag-processor About a minute ago    Up 29 seconds (healthy)
8080/tcp
datacross_airflow-airflow-scheduler-1 datacross-airflow:3.3.1            "/usr/bin/dumb-init ..."          airflow-scheduler    About a minute ago    Up 29 seconds (healthy)
8080/tcp
datacross_airflow-airflow-triggerer-1 datacross-airflow:3.3.1            "/usr/bin/dumb-init ..."          airflow-triggerer     About a minute ago    Up 29 seconds (healthy)
8080/tcp
datacross_airflow-airflow-worker-1 datacross-airflow:3.3.1            "/usr/bin/dumb-init ..."          airflow-worker        About a minute ago    Up 8 seconds (health: starting)
8080/tcp
datacross_airflow-postgres-1         postgres:16                         "docker-entrypoint.s..."          postgres             About a minute ago    Up About a minute (healthy)
5432/tcp
datacross_airflow-redis-1            redis:7.2-bookworm                 "docker-entrypoint.s..."          redis                About a minute ago    Up About a minute (healthy)
6379/tcp
PS C:\DataCross_Airflow> docker compose exec airflow-worker python -c "import pandas, openpyxl, oracledb; print('DEPENDENCIAS DATACROSS: OK!')"
DEPENDENCIAS DATACROSS: OK
PS C:\DataCross_Airflow> docker compose exec airflow-worker ls -lh /opt/airflow/data
total 2.7M
-rwxrwxrwx 1 airflow root 2.7M Aug 28 17:04 dataset_dados_e_danados.xlsx
PS C:\DataCross_Airflow> docker compose exec airflow-worker python -c "import pandas as pd; df=pd.read_excel('/opt/airflow/data/dataset_dados_e_danados.xlsx'); print('LINHAS:', len(df)); print('COLUNAS:', len(df.columns))"
LINHAS: 10000
COLUNAS: 64
PS C:\DataCross_Airflow>

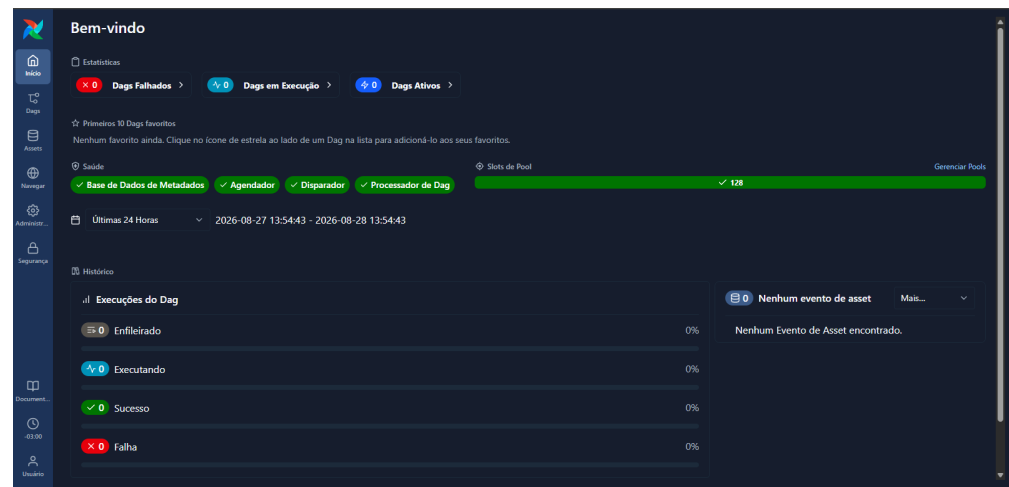
```

PRINT 03 - Containers do Airflow em execução, imagem personalizada datacross-airflow:3.3.1, dependências Python validadas e leitura do dataset com 10.000 linhas e 64 colunas.

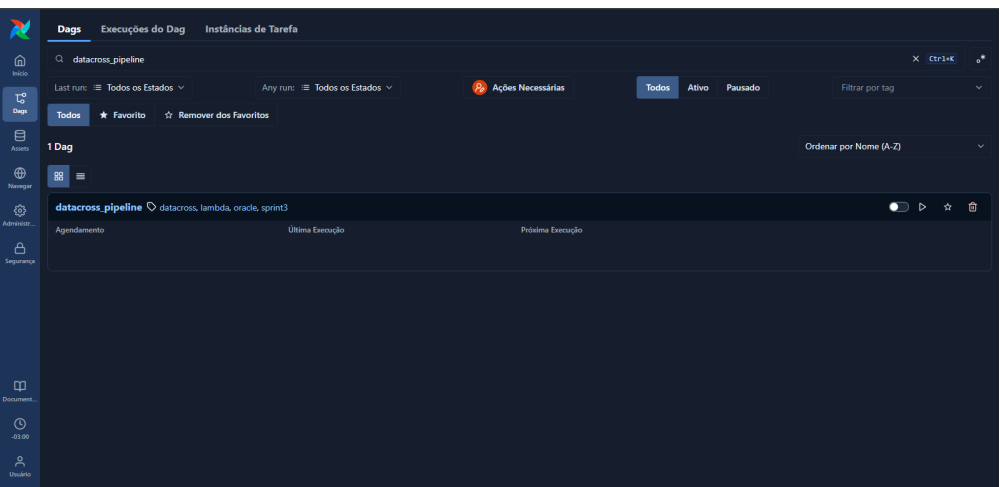
9. Evidências de integração Airflow -> Oracle e registro da DAG

```
PS C:\DataCross_Airflow> docker compose exec airflow-worker python -c "import os, oracledb; conn=oracledb.connect(user=os.environ['ORACLE_USER'], password=os.environ['ORACLE_PASSWORD'], dsn=os.environ['ORACLE_DSN'], config_dir='/opt/airflow/wallet', wallet_location='/opt/airflow/wallet', wallet_password=os.environ['ORACLE_WALLET_PASSWORD']); cur=conn.cursor(); cur.execute('SELECT USER, SYSDATE FROM dual'); print('CONEXAO ORACLE DATACROSS: OK'); print(cur.fetchone()); cur.close(); conn.close()"
CONEXAO ORACLE DATACROSS: OK
('ADMIN', datetime.datetime(2026, 8, 28, 17, 30, 15))
PS C:\DataCross_Airflow> |
```

PRINT 04 - Conexão do airflow-worker com o Oracle Autonomous Database 26 AI via Wallet: CONEXAO ORACLE DATACROSS: OK.

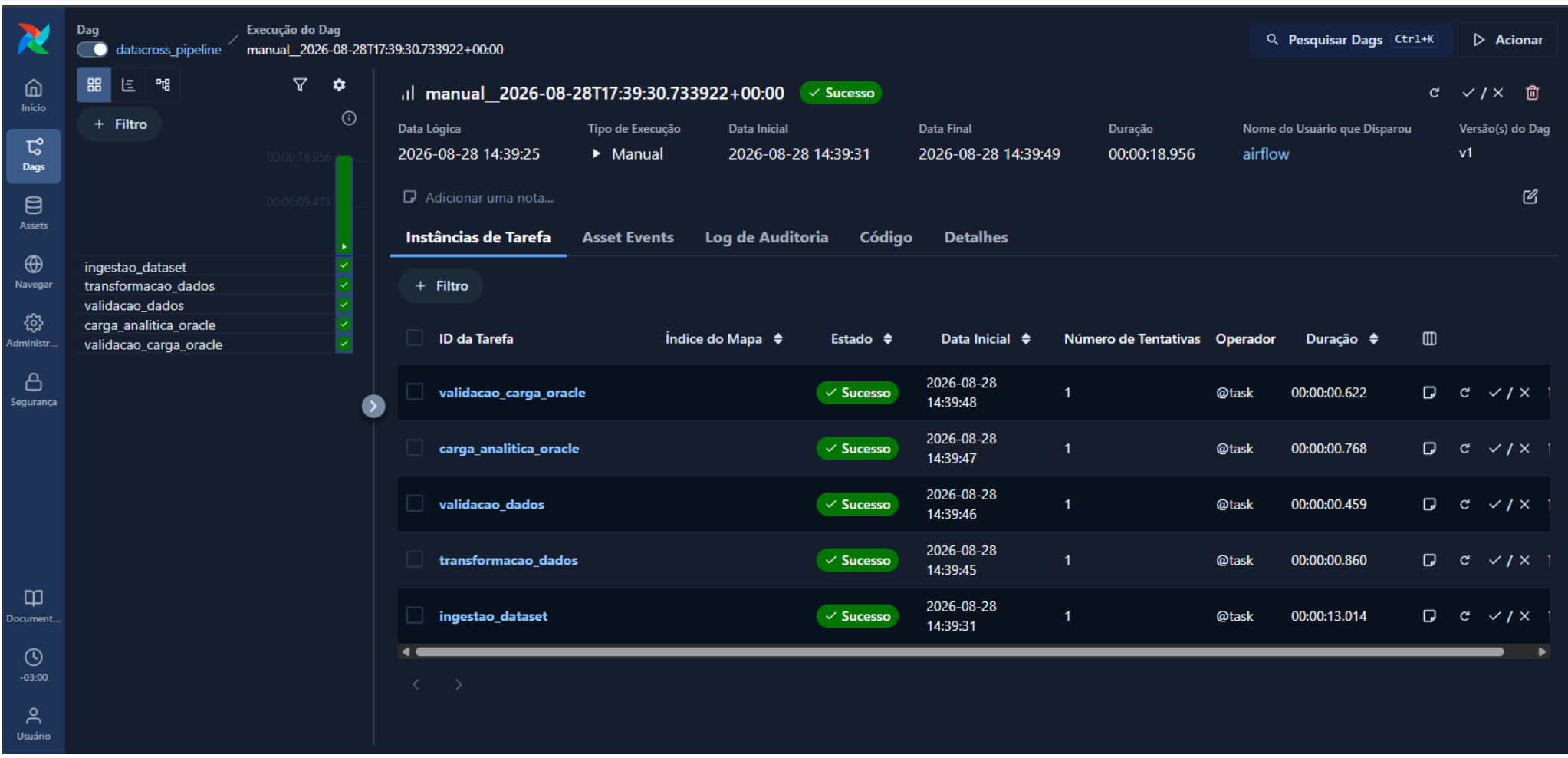


PRINT 05 - Interface do Apache Airflow operacional e componentes de saúde disponíveis.



PRINT 06 - DAG datacross_pipeline registrada com tags datacross, lambda, oracle e sprint3.

10. Evidências de execução, logs e validação



PRINT 07 - Execução manual concluída com status Sucesso. As cinco tarefas da DAG finalizaram com sucesso em uma única tentativa.

A evidência demonstra o encadeamento funcional exigido: ingestão, transformação, validação e carga analítica, acrescido de uma etapa de validação pós-carga para reforçar a integridade do pipeline.

10.1. Validação pré-carga

validacao_dados

Sucesso

Operador

@task

Data Inicial

2026-08-28 14:39:46

Data Final

2026-08-28 14:39:47

Duração

00:00:00.459

Versão do Dag

v1

Adicionar uma nota...

Logs

Modelos Renderizados

Storage

Asset Events

Log de Auditoria

Código

Detalhes

Todos os Níveis de Log

Todas as Fontes

Search logs...

Log message source details

Pre Execute

6

[2026-08-28 14:39:47]

INFO

- Current task name:validacao_dados

7

[2026-08-28 14:39:47]

INFO

- Dag name:datacross_pipeline

8

[2026-08-28 14:39:47]

INFO

- ===== DATACROSS | VALIDAÇÃO =====

9

[2026-08-28 14:39:47]

INFO

- [VALIDAÇÃO] dataset_nao_vazio : OK

10

[2026-08-28 14:39:47]

INFO

- [VALIDAÇÃO] total_origem_preservado : OK

11

[2026-08-28 14:39:47]

INFO

- [VALIDAÇÃO] chaves_sem_nulos : OK

12

[2026-08-28 14:39:47]

INFO

- [VALIDAÇÃO] internados_consistentes : OK

13

[2026-08-28 14:39:47]

INFO

- [VALIDAÇÃO] permanencia_nao_negativa : OK

14

[2026-08-28 14:39:47]

INFO

- [VALIDAÇÃO] ocupacao_intervalo_valido : OK

15

[2026-08-28 14:39:47]

INFO

- [VALIDAÇÃO] sem_duplicidade_granularidade : OK

16

[2026-08-28 14:39:47]

INFO

- [VALIDAÇÃO] Linhas na origem: 10000

17

[2026-08-28 14:39:47]

INFO

- [VALIDAÇÃO] Soma das internações agregadas: 10000

18

[2026-08-28 14:39:47]

INFO

- [VALIDAÇÃO] Registros analíticos: 1102

19

[2026-08-28 14:39:47]

INFO

- [VALIDAÇÃO] Pipeline aprovado para carga analítica.

20

[2026-08-28 14:39:47]

INFO

- VALIDAÇÃO: OK

21

[2026-08-28 14:39:47]

INFO

- Done. Returned value was: {'processed_path': '/opt/airflow/data/processed/indicadores_hospitalares.csv', 'validation_path': '/opt/airflow/d

Post Execute

PRINT 08 - Logs da tarefa validacao_dados: sete regras de qualidade aprovadas, 10.000 linhas preservadas e 1.102 registros analíticos gerados.

A carga é interrompida automaticamente caso qualquer check retorne FALHA. Nesta execução, todas as regras foram aprovadas antes do envio ao Oracle.

10.2. Carga analítica no Oracle

carga_analitica_oracle

✓ Sucesso

Operador

@task

Data Inicial

2026-08-28 14:39:47

Data Final

2026-08-28 14:39:48

Duração

00:00:00.768

Versão do Dag

v1

Adicionar uma nota...

Logs

Modelos Renderizados

Storage

Asset Events

Log de Auditoria

Código

Detalhes

Todos os Níveis de Log

Todas as Fontes

Search logs...

Log message source details

Pre Execute

8 [2026-08-28 14:39:48] INFO - ===== DATACROSS | CARGA ANALÍTICA =====

9 [2026-08-28 14:39:48] INFO - Tabela T_INDICADORES_AIRFLOW criada no Oracle.

10 [2026-08-28 14:39:48] INFO - Registros carregados em T_INDICADORES_AIRFLOW: 1102

11 [2026-08-28 14:39:48] INFO - Estratégia de carga: FULL REFRESH idempotente

12 [2026-08-28 14:39:48] INFO - CARGA ANALÍTICA ORACLE: OK

13 [2026-08-28 14:39:48] INFO - Done. Returned value was: {'tabela': 'T_INDICADORES_AIRFLOW', 'registros_carregados': 1102, 'linhas_origem': 10000}

Post Execute

PRINT 09 - Log da tarefa carga_analitica_oracle: criação da T_INDICADORES_AIRFLOW, carga de 1.102 registros e estratégia FULL REFRESH idempotente.

10.3. Validação pós-carga

 validacao_carga_oracle

✓ Sucesso

↺ ✓ / ✕

Operador	Data Inicial	Data Final	Duração	Versão do Dag
@task	2026-08-28 14:39:48	2026-08-28 14:39:49	00:00:00.622	v1

 Adicionar uma nota...



Logs

Modelos Renderizados

Storage

Asset Events

Log de Auditoria

Código

Detalhes

Todos os Níveis de Log

Todas as Fontes

▶ Log message source details

▶ Pre Execute

8 [2026-08-28 14:39:49] INFO - ===== DATACROSS | VALIDAÇÃO PÓS-CARGA =====

9 [2026-08-28 14:39:49] INFO - [PÓS-CARGA] Registros analíticos Oracle: 1102

10 [2026-08-28 14:39:49] INFO - [PÓS-CARGA] Soma TOTAL_INTERNAÇÕES: 10000

11 [2026-08-28 14:39:49] INFO - [PÓS-CARGA] Soma TOTAL_INTERNADOS: 655

12 [2026-08-28 14:39:49] INFO - [PÓS-CARGA] Integridade entre origem e Oracle: OK

13 [2026-08-28 14:39:49] INFO - PIPELINE DATACROSS CONCLUÍDO COM SUCESSO.

14 [2026-08-28 14:39:49] INFO - Done. Returned value was: {'status': 'SUCESSO', 'tabela': 'T_INDICADORES_AIRFLOW', 'registros_analiticos': 1102, 'total_internacoes': 10000}

▶ Post Execute

18 [2026-08-28 14:39:49] INFO - Task operator:<Task(_PythonDecoratedOperator): validacao_carga_oracle>

PRINT 10 - Reconciliação pós-carga: 1.102 registros analíticos, soma de 10.000 internações, 655 internados e integridade origem x Oracle = OK.

Evidência de qualidade

A validação pós-carga evita tratar um simples commit como prova de sucesso. O pipeline confirma o estado final no banco de destino e compara os totais com a origem antes de encerrar.

10.4. Validação independente no SQL Developer

The screenshot shows the SQL Developer interface with a query window titled 'DADOS_E_DANADOS' and a worksheet titled 'T_INDICADORES_AIRFLOW'. The query is as follows:

```
SELECT
COUNT(*) AS REGISTROS_ANALITICOS,
SUM(TOTAL_INTERNAÇOES) AS TOTAL_INTERNAÇOES,
SUM(TOTAL_INTERNADOS) AS TOTAL_INTERNADOS
FROM T_INDICADORES_AIRFLOW;
```

The 'Script Output' window shows the results of the query, which are displayed in a table format. The table has three columns: 'REGISTROS_ANALITICOS', 'TOTAL_INTERNAÇOES', and 'TOTAL_INTERNADOS'. The first row shows the counts: 1102, 10000, and 655 respectively. Below this, there are two sections of data, each with a header row and a list of rows.

REGISTROS_ANALITICOS	TOTAL_INTERNAÇOES	TOTAL_INTERNADOS
1102	10000	655

NOME_HOSPITAL	DOENÇA_DETECTADA
hospital_do_coracao	Pneumonia
hospital_geral_alegria	Pneumonia
hospital_regional_esperanca	Pneumonia
hospital_santa_maria	Pneumonia
hospital_sao_jose	Pneumonia
hospital_sao_lucas	Pneumonia
hospital_viva_bem	Pneumonia
hospital_boa_viagem	Pneumonia
hospital_metropolitano	Pneumonia
hospital_das_nacoes	Pneumonia
hospital_nossa_senhora_da_paz	Pneumonia

NOME_HOSPITAL	DOENÇA_DETECTADA
hospital_central_da_cidade	Pneumonia
hospital_universitario_sr._dolphin_sakila	Pneumonia
hospital_universitario_sr._dolphin_sakila	Infecção do trato urinário (ITU)
hospital_das_clinicas	Pneumonia
hospital_do_trabalhador	Pneumonia
hospital_universitario_sr._dolphin_sakila	avc_isquemico
hospital_regional_esperanca	Infecção do trato urinário (ITU)
hospital_geral_alegria	Infecção do trato urinário (ITU)

PRINT 11 - Consulta direta no SQL Developer confirma 1.102 registros analíticos, 10.000 internações e 655 internados na T_INDICADORES_AIRFLOW.

Essa evidência valida a persistência fora do contexto do Airflow, demonstrando que os dados estão efetivamente disponíveis no Oracle Autonomous Database para consumo analítico.

11. Qualidade, data mart e consumo analítico

Regra de qualidade	Objetivo	Resultado observado
dataset_nao_vazio	Impedir processamento sem dados	OK
total_origem_preservado	Garantir que a agregação represente todas as interações	10.000 = 10.000
chaves_sem_nulos	Evitar registros sem hospital/competência/diagnóstico	OK
internados_consistentes	Garantir TOTAL_INTERNADOS <= TOTAL_INTERNACOES	OK
permanencia_nao_negativa	Bloquear permanência média negativa	OK
ocupacao_intervalo_valido	Garantir taxa entre 0 e 1	OK
sem_duplicidade_granularidade	Garantir unicidade hospital + competência + diagnóstico	OK

11.1 Estrutura analítica disponibilizada

Campo	Uso analítico
ID_HOSPITAL / NOME_HOSPITAL	Segmentação e comparação entre unidades
COMPETENCIA	Evolução temporal mensal
DOENCA_DETECTADA	Perfil clínico: AVC, ITU e pneumonia
TOTAL_INTERNACOES / TOTAL_INTERNADOS	Volume e pressão assistencial
MEDIA_PERMANENCIA_DIAS	Rotatividade e permanência hospitalar
MEDIA_LEITOS_DISPONIVEIS / MEDIA_TAXA_OCUPACAO	Capacidade e ocupação
CUSTO_ESTIMADO_TOTAL	Estimativa analítica baseada em diária x permanência; não é custo contábil real

12. Código da DAG - trechos essenciais

O código completo e executável da DAG está incorporado ao Anexo A deste próprio relatório. Abaixo são reproduzidos os trechos essenciais que demonstram a definição da DAG, as dependências e a integração com o Oracle.

Definição da DAG e ingestão

```
@dag(
    dag_id="datacross_pipeline",
    description="Pipeline batch DataCross: ingestão, transformação, validação e carga
        analítica no Oracle 26 AI",
    schedule=None,
    start_date=datetime(2026, 8, 1, tzinfo=timezone.utc),
    catchup=False,
    tags=["datacross", "sprint3", "lambda", "oracle"],
)
def datacross_pipeline():

    @task(task_id="ingestao_dataset")
    def ingestao_dataset() -> dict:
        """
        Camada de ingestão:
        lê o XLSX sintético e persiste uma cópia bruta em JSON Lines.
        """
        LOGGER.info("===== DATACROSS | INGESTÃO =====")
        LOGGER.info("Arquivo de entrada: %s", INPUT_FILE)

        if not INPUT_FILE.exists():
            raise FileNotFoundError(
                f"Dataset não encontrado em {INPUT_FILE}. "
                "Verifique o volume /opt/airflow/data."
            )

        RAW_DIR.mkdir(parents=True, exist_ok=True)
        PROCESSED_DIR.mkdir(parents=True, exist_ok=True)

        df = pd.read_excel(INPUT_FILE)

        if df.empty:
            raise ValueError("0 dataset foi carregado, porém está vazio.")

        missing_columns = sorted(REQUIRED_COLUMNS.difference(df.columns))
        if missing_columns:
            raise ValueError(
                "Colunas obrigatórias ausentes no dataset: "
                + ", ".join(missing_columns)
            )

        df.to_json(
            RAW_FILE,
            orient="records",
            lines=True,
            force_ascii=False,
            date_format="iso",
        )

        LOGGER.info("Dataset lido com sucesso.")
        LOGGER.info("Total de linhas recebidas: %s", len(df))
        LOGGER.info("Total de colunas recebidas: %s", len(df.columns))
```

```

LOGGER.info("Cópia bruta persistida em: %s", RAW_FILE)
LOGGER.info("INGESTÃO: OK")

return {
    "raw_path": str(RAW_FILE),
    "linhas_origem": int(len(df)),
    "colunas_origem": int(len(df.columns)),
}

```

Gate de validação

```

@task(task_id="validacao_dados")
def validacao_dados(info_transformacao: dict) -> dict:
    """
    Camada de qualidade:
    impede a carga se as regras essenciais de integridade falharem.
    """
    LOGGER.info("===== DATACROSS | VALIDAÇÃO =====")

    df = pd.read_csv(
        info_transformacao["processed_path"],
        parse_dates=["competencia"],
    )

    expected_total = int(info_transformacao["linhas_origem"])
    aggregated_total = int(df["total_internacoes"].sum())

    checks = {
        "dataset_nao_vazio": len(df) > 0,
        "total_origem_preservado": aggregated_total == expected_total,
        "chaves_sem_nulos": not df[
            [
                "id_do_hospital",
                "nome_do_hospital",
                "competencia",
                "doenca_detectada",
            ]
        ].isna().any().any(),
        "internados_consistentes": bool(
            (df["total_internados"] <= df["total_internacoes"]).all()
        ),
        "permanencia_nao_negativa": bool(
            (df["media_permanencia_dias"] >= 0).all()
        ),
        "ocupacao_intervalo_valido": bool(
            df["media_taxa_ocupacao"].between(0, 1).all()
        ),
        "sem_duplicidade_granularidade": not df.duplicated(
            subset=[
                "id_do_hospital",
                "competencia",
                "doenca_detectada",
            ]
        ).any(),
    }

    for check_name, result in checks.items():
        LOGGER.info(
            "[VALIDAÇÃO] %-35s : %s",
            check_name,
            "OK" if result else "FALHA",
        )

    LOGGER.info(

```

```

        "[VALIDAÇÃO] Linhas na origem: %s",
        expected_total,
    )
    LOGGER.info(
        "[VALIDAÇÃO] Soma das internações agregadas: %s",
        aggregated_total,
    )
    LOGGER.info(
        "[VALIDAÇÃO] Registros analíticos: %s",
        len(df),
    )

    failed = [name for name, result in checks.items() if not result]
    if failed:
        raise ValueError(
            "Pipeline reprovado nas validações: " + ", ".join(failed)
        )

    validation_payload = {
        "status": "APROVADO",
        "linhas_origem": expected_total,
        "internacoes_agregadas": aggregated_total,
        "registros_analiticos": int(len(df)),
        "checks": checks,
    }

    with open(VALIDATION_FILE, "w", encoding="utf-8") as file:
        json.dump(
            validation_payload,
            file,
            ensure_ascii=False,
            indent=2,
        )

    LOGGER.info("[VALIDAÇÃO] Pipeline aprovado para carga analítica.")
    LOGGER.info("VALIDAÇÃO: OK")

    return {
        "processed_path": info_transformacao["processed_path"],
        "validation_path": str(VALIDATION_FILE),
        "linhas_origem": expected_total,
        "registros_analiticos": int(len(df)),
    }

```

Carga analítica no Oracle

```

@task(task_id="carga_analitica_oracle")
def carga_analitica_oracle(info_validacao: dict) -> dict:
    """
    Serving Layer:
    executa carga full-refresh do data mart no Oracle Autonomous 26 AI.
    """
    LOGGER.info("===== DATACROSS | CARGA ANALÍTICA =====")

    df = pd.read_csv(
        info_validacao["processed_path"],
        parse_dates=["competencia"],
    )

    conn = _oracle_connection()
    cur = conn.cursor()

    create_table_sql = f"""
    CREATE TABLE {ORACLE_TABLE} (

```

```

        ID_HOSPITAL NUMBER(10) NOT NULL,
        NOME_HOSPITAL VARCHAR2(120) NOT NULL,
        COMPETENCIA DATE NOT NULL,
        DOENCA_DETECTADA VARCHAR2(100) NOT NULL,
        TOTAL_INTERNAÇOES NUMBER(10) NOT NULL,
        TOTAL_INTERNADOS NUMBER(10) NOT NULL,
        MEDIA_PERMANENCIA_DIAS NUMBER(10,2),
        MEDIA_LEITOS_DISPONIVEIS NUMBER(10,2),
        MEDIA_TAXA_OCUPACAO NUMBER(10,6),
        CUSTO_ESTIMADO_TOTAL NUMBER(16,2),
        DT_CARGA TIMESTAMP DEFAULT SYSTIMESTAMP,
        CONSTRAINT PK_T_INDICADORES_AIRFLOW
            PRIMARY KEY (
                ID_HOSPITAL,
                COMPETENCIA,
                DOENCA_DETECTADA
            )
    )
"""

try:
    cur.execute(create_table_sql)
    LOGGER.info("Tabela %s criada no Oracle.", ORACLE_TABLE)
except oracledb.DatabaseError as exc:
    error_obj = exc.args[0]
    if getattr(error_obj, "code", None) == 955:
        LOGGER.info(
            "Tabela %s já existe; será reutilizada.",
            ORACLE_TABLE,
        )
    else:
        cur.close()
        conn.close()
        raise

# Full refresh controlado para tornar a DAG idempotente.
cur.execute(f"DELETE FROM {ORACLE_TABLE}")

insert_sql = f"""
INSERT INTO {ORACLE_TABLE} (
    ID_HOSPITAL,
    NOME_HOSPITAL,
    COMPETENCIA,
    DOENCA_DETECTADA,
    TOTAL_INTERNAÇOES,
    TOTAL_INTERNADOS,
    MEDIA_PERMANENCIA_DIAS,
    MEDIA_LEITOS_DISPONIVEIS,
    MEDIA_TAXA_OCUPACAO,
    CUSTO_ESTIMADO_TOTAL
)
VALUES (
    :1, :2, :3, :4, :5,
    :6, :7, :8, :9, :10
)
"""

rows = []
for row in df.itertuples(index=False):
    rows.append(
        (
            int(row.id_do_hospital),
            str(row.nome_do_hospital),
            row.competencia.to_pydatetime(),

```

```

        str(row.doenca_detectada),
        int(row.total_internacoes),
        int(row.total_internados),
        float(row.media_permanencia_dias),
        float(row.media_leitos_disponiveis),
        float(row.media_taxa_ocupacao),
        float(row.custo_estimado_total),
    )
)

cur.executemany(insert_sql, rows)
conn.commit()

LOGGER.info(
    "Registros carregados em %s: %s",
    ORACLE_TABLE,
    len(rows),
)
LOGGER.info("Estratégia de carga: FULL REFRESH idempotente")
LOGGER.info("CARGA ANALÍTICA ORACLE: OK")

cur.close()
conn.close()

return {
    "tabela": ORACLE_TABLE,
    "registros_carregados": int(len(rows)),
    "linhas_origem": int(info_validacao["linhas_origem"]),
}

```

Validação pós-carga e dependências

```

@task(task_id="validacao_carga_oracle")
def validacao_carga_oracle(info_carga: dict) -> dict:
    """
    Validação pós-carga:
    compara Oracle com os totais aprovados antes da carga.
    """
    LOGGER.info(
        "===== DATACROSS | VALIDAÇÃO PÓS-CARGA ====="
    )

    conn = _oracle_connection()
    cur = conn.cursor()

    cur.execute(
        f"""
        SELECT
            COUNT(*) AS REGISTROS_ANALITICOS,
            SUM(TOTAL_INTERNAcoes) AS TOTAL_INTERNAcoes,
            SUM(TOTAL_INTERNADOS) AS TOTAL_INTERNADOS
        FROM {ORACLE_TABLE}
        """
    )

    registros, total_internacoes, total_internados = cur.fetchone()

    cur.close()
    conn.close()

    registros = int(registros or 0)
    total_internacoes = int(total_internacoes or 0)
    total_internados = int(total_internados or 0)

```

```
if registros != int(info_carga["registros_carregados"]):
    raise ValueError(
        "Quantidade de registros no Oracle difere da carga realizada."
    )

if total_internacoes != int(info_carga["linhas_origem"]):
    raise ValueError(
        "Total de internações no Oracle não corresponde à origem."
    )

LOGGER.info(
    "[PÓS-CARGA] Registros analíticos Oracle: %s",
    registros,
)
LOGGER.info(
    "[PÓS-CARGA] Soma TOTAL_INTERNACOES: %s",
    total_internacoes,
)
LOGGER.info(
    "[PÓS-CARGA] Soma TOTAL_INTERNADOS: %s",
    total_internados,
)
LOGGER.info("[PÓS-CARGA] Integridade entre origem e Oracle: OK")
LOGGER.info("PIPELINE DATACROSS CONCLUÍDO COM SUCESSO.")

return {
    "status": "SUCESSO",
    "tabela": ORACLE_TABLE,
    "registros_analiticos": registros,
    "total_internacoes": total_internacoes,
    "total_internados": total_internados,
}

ingestao = ingestao_dataset()
transformacao = transformacao_dados(ingestao)
validacao = validacao_dados(transformacao)
carga = carga_analitica_oracle(validacao)
validacao_carga_oracle(carga)
```

Entrega do código

O código integral do arquivo datacross_pipeline.py está incorporado ao Anexo A deste relatório, permitindo execução, inspeção e avaliação independente em um único arquivo.

13. Como a arquitetura apoia o Challenge Oracle

A proposta transforma dados hospitalares de origem em um produto analítico validado e rastreável. O Airflow separa responsabilidades, aplica gates de qualidade e reduz a dependência de processamento manual. O Oracle Autonomous Database centraliza a camada de serving, permitindo que a mesma informação seja consumida por consultas analíticas e pelo painel DataCross.

Capacidade do pipeline	Contribuição para o DataCross
Orquestração reprodutível	Executa a mesma sequência de ingestão, transformação, validação e carga sem depender de etapas manuais desconectadas.
Qualidade antes da carga	Evita propagar dados estruturalmente inválidos para o repositório analítico.
Reconciliação pós-carga	Comprova que o destino preservou os volumes esperados antes de declarar sucesso.
Data mart hospitalar	Entrega indicadores por hospital, período e diagnóstico para apoiar análise de permanência, ocupação e pressão assistencial.
Evolução para streaming	A Speed Layer permite incorporar eventos FHIR/API sem descartar o processamento histórico já implementado.

Princípio de uso

O DataCross é uma camada de apoio à decisão e não substitui validação clínica ou regulatória. A arquitetura fornece dados e indicadores para priorização e análise, mantendo a decisão final com profissionais responsáveis.

14. Limitações do MVP e evolução arquitetural

Limitação atual	Tratamento no projeto / evolução
Base sintética	Utilizada para validação técnica. Em produção, o pipeline deve receber dados autorizados e governados de sistemas hospitalares.
Speed Layer não implantada	Representada conceitualmente; a implementação futura pode usar FHIR/API e serviços OCI adequados a eventos.
Raw zone local no laboratório	O volume Docker é suficiente para a Sprint; em produção, Oracle Object Storage é a opção coerente para desacoplar ingestão e processamento.
Full Refresh	Adequado ao pequeno volume do MVP e torna o processo idempotente; volumes maiores podem evoluir para carga incremental/merge.
Indicadores não clínicos prescritivos	O pipeline produz informação analítica; não automatiza alta/transferência sem validação humana.

14.1 Roadmap técnico

- Adicionar ingestão autenticada de recursos FHIR e eventos de movimentação hospitalar.
- Migrar a raw zone de laboratório para Oracle Object Storage com versionamento e retenção.
- Introduzir carga incremental com watermark/competência quando o volume justificar.
- Publicar métricas do pipeline (latência, falhas, registros rejeitados, tempo de carga) para observabilidade.
- Conectar a camada serving ao painel DataCross e às consultas Select AI já validadas na disciplina de segunda-feira.

15. Conclusão e explicação executiva final

15.1 Checklist de atendimento

Item obrigatório	Status
Arquitetura Lambda ou Kappa definida e justificada	ATENDIDO - Lambda
Volume, batch, streaming, painel e Oracle considerados	ATENDIDO
Diagrama da arquitetura	ATENDIDO - Figura 1
DAG Airflow com ingestão, transformação, validação e carga	ATENDIDO
Código da DAG	ATENDIDO - Seção 12 + Anexo A
Evidência visual da DAG e execução	ATENDIDO - PRINTS 06-07
Logs e validação do pipeline	ATENDIDO - PRINTS 08-10
Camadas conceituais descritas	ATENDIDO - Figura 2 e Seção 5
Integração com Oracle Database 26 AI	ATENDIDO - PRINTS 04, 09, 10 e 11
Participantes em ordem alfabética	ATENDIDO - capa
Nome do arquivo conforme RM do representante	ATENDIDO - evidencias-sprint3-rm570626.pdf

15.2 Explicação executiva final - máximo 5 linhas

O pipeline DataCross transforma dados hospitalares em indicadores analíticos validados.
O Airflow automatiza ingestão, transformação, validação e carga, com rastreabilidade.
O Oracle Autonomous Database 26 AI centraliza o data mart para consultas e painel.
A Arquitetura Lambda mantém o batch atual e permite evolução futura para FHIR/API.
Assim, a solução reduz esforço manual e aumenta a confiabilidade dos insights de gestão.

Resultado: a Sprint demonstra uma arquitetura tecnicamente coerente, um pipeline funcional executado de ponta a ponta e evidências independentes de qualidade e persistência no Oracle.

Anexo A. Código completo da DAG datacross_pipeline.py

O conteúdo abaixo corresponde ao código integral executado no Apache Airflow durante a Sprint 3. Ele está incorporado neste PDF para que a entrega em arquivo único contemple explicitamente a exigência de entrega do código da DAG.

```
001 from __future__ import annotations
002
003 import json
004 import logging
005 import os
006 from datetime import datetime, timezone
007 from pathlib import Path
008
009 import oracledb
010 import pandas as pd
011 from airflow.sdk import dag, task
012
013
014 # -----
015 # Configuração geral do pipeline DataCross
016 # -----
017
018 LOGGER = logging.getLogger(__name__)
019
020 DATA_DIR = Path("/opt/airflow/data")
021 INPUT_FILE = DATA_DIR / "dataset_dados_e_danados.xlsx"
022 RAW_DIR = DATA_DIR / "raw"
023 PROCESSED_DIR = DATA_DIR / "processed"
024
025 RAW_FILE = RAW_DIR / "prontuarios_raw.jsonl"
026 PROCESSED_FILE = PROCESSED_DIR / "indicadores_hospitales.csv"
027 VALIDATION_FILE = PROCESSED_DIR / "validacao_pipeline.json"
028
029 ORACLE_TABLE = "T_INDICADORES_AIRFLOW"
030
031 REQUIRED_COLUMNS = {
032     "id_do_hospital",
033     "nome_do_hospital",
034     "data_de_internacao",
035     "status",
036     "doenca_detectada",
037     "tempo_de_permanencia_dias",
038     "numero_de_leitos_disponiveis",
039     "taxa_ocupacao_avc_itu_pneumonia",
040     "valor_diaria_leito",
041 }
042
043
044 def _oracle_connection():
045     """Cria conexão mTLS com o Oracle Autonomous Database usando o Wallet."""
046     required_env = [
047         "ORACLE_USER",
048         "ORACLE_PASSWORD",
049         "ORACLE_DSN",
050         "ORACLE_WALLET_PASSWORD",
051     ]
052
053     missing = [name for name in required_env if not os.getenv(name)]
054     if missing:
055         raise RuntimeError(
056             "Variáveis de ambiente Oracle ausentes: " + ", ".join(missing)
057         )
058
059     return oracledb.connect(
060         user=os.environ["ORACLE_USER"],
061         password=os.environ["ORACLE_PASSWORD"],
062         dsn=os.environ["ORACLE_DSN"],
063         config_dir="/opt/airflow/wallet",
064         wallet_location="/opt/airflow/wallet",
065         wallet_password=os.environ["ORACLE_WALLET_PASSWORD"],
```

Anexo A - continuação

```

066     )
067
068
069 @dag(
070     dag_id="datacross_pipeline",
071     description="Pipeline batch DataCross: ingestão, transformação, validação e carga analítica no Oracle 26 AI",
072     schedule=None,
073     start_date=datetime(2026, 8, 1, tzinfo=timezone.utc),
074     catchup=False,
075     tags=["datacross", "sprint3", "lambda", "oracle"],
076 )
077 def datacross_pipeline():
078
079     @task(task_id="ingestao_dataset")
080     def ingestao_dataset() -> dict:
081         """
082         Camada de ingestão:
083         lê o XLSX sintético e persiste uma cópia bruta em JSON Lines.
084         """
085         LOGGER.info("===== DATACROSS | INGESTÃO =====")
086         LOGGER.info("Arquivo de entrada: %s", INPUT_FILE)
087
088         if not INPUT_FILE.exists():
089             raise FileNotFoundError(
090                 f"Dataset não encontrado em {INPUT_FILE}. "
091                 "Verifique o volume /opt/airflow/data."
092             )
093
094         RAW_DIR.mkdir(parents=True, exist_ok=True)
095         PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
096
097         df = pd.read_excel(INPUT_FILE)
098
099         if df.empty:
100             raise ValueError("0 dataset foi carregado, porém está vazio.")
101
102         missing_columns = sorted(REQUIRED_COLUMNS.difference(df.columns))
103         if missing_columns:
104             raise ValueError(
105                 "Colunas obrigatórias ausentes no dataset: "
106                 + ", ".join(missing_columns)
107             )
108
109         df.to_json(
110             RAW_FILE,
111             orient="records",
112             lines=True,
113             force_ascii=False,
114             date_format="iso",
115         )
116
117         LOGGER.info("Dataset lido com sucesso.")
118         LOGGER.info("Total de linhas recebidas: %s", len(df))
119         LOGGER.info("Total de colunas recebidas: %s", len(df.columns))
120         LOGGER.info("Cópia bruta persistida em: %s", RAW_FILE)
121         LOGGER.info("INGESTÃO: OK")
122
123         return {
124             "raw_path": str(RAW_FILE),
125             "linhas_origem": int(len(df)),
126             "colunas_origem": int(len(df.columns)),
127         }
128
129     @task(task_id="transformacao_dados")
130     def transformacao_dados(info_ingestao: dict) -> dict:

```

Anexo A - continuação

```

131     """
132     Camada de transformação:
133     padroniza campos e cria um data mart agregado por
134     hospital + competência mensal + diagnóstico.
135     """
136     LOGGER.info("===== DATACROSS | TRANSFORMAÇÃO =====")
137
138     raw_path = Path(info_ingestao["raw_path"])
139     df = pd.read_json(raw_path, lines=True)
140
141     # Padronização textual
142     df["status"] = df["status"].astype(str).str.strip().str.lower()
143     df["doenca_detectada"] = (
144         df["doenca_detectada"].astype(str).str.strip()
145     )
146     df["nome_do_hospital"] = (
147         df["nome_do_hospital"].astype(str).str.strip()
148     )
149
150     # Conversão de datas
151     df["data_de_internacao"] = pd.to_datetime(
152         df["data_de_internacao"],
153         dayfirst=True,
154         errors="coerce",
155     )
156
157     if df["data_de_internacao"].isna().any():
158         qtd = int(df["data_de_internacao"].isna().sum())
159         raise ValueError(
160             f"Foram encontradas {qtd} datas de internação inválidas."
161         )
162
163     # Competência mensal para granularidade analítica.
164     df["competencia"] = (
165         df["data_de_internacao"]
166         .dt.to_period("M")
167         .dt.to_timestamp()
168     )
169
170     # Conversão da diária (ex.: "R$ 150" -> 150.00)
171     diaria_txt = (
172         df["valor_diaria_leito"]
173         .astype(str)
174         .str.replace(r"^[0-9,.\-]", "", regex=True)
175         .str.replace(",", ".", regex=False)
176     )
177     df["valor_diaria_numerico"] = pd.to_numeric(
178         diaria_txt, errors="coerce"
179     )
180
181     if df["valor_diaria_numerico"].isna().any():
182         qtd = int(df["valor_diaria_numerico"].isna().sum())
183         raise ValueError(
184             f"Foram encontradas {qtd} diárias de leito não conversíveis."
185         )
186
187     numeric_columns = [
188         "tempo_de_permanencia_dias",
189         "numero_de_leitos_disponiveis",
190         "taxa_ocupacao_avc_itu_pneumonia",
191     ]
192     for column in numeric_columns:
193         df[column] = pd.to_numeric(df[column], errors="coerce")
194
195     if df[numeric_columns].isna().any().any():

```

Anexo A - continuação

```
196         invalidos = df[numeric_columns].isna().sum()
197         invalidos = invalidos[invalidos > 0].to_dict()
198         raise ValueError(
199             f"Valores numéricos inválidos encontrados: {invalidos}"
200         )
201
202     # Métrica de custo estimado. Não representa custo contábil real.
203     df["custo_estimado"] = (
204         df["valor_diaria_numerico"]
205         * df["tempo_de_permanencia_dias"]
206     )
207
208     # Indicador binário para pacientes ainda internados.
209     df["flag_internado"] = (df["status"] == "internado").astype(int)
210
211     indicadores = (
212         df.groupby(
213             [
214                 "id_do_hospital",
215                 "nome_do_hospital",
216                 "competencia",
217                 "doenca_detectada",
218             ],
219             as_index=False,
220         )
221         .agg(
222             total_internacoes=("id_do_hospital", "size"),
223             total_internados=("flag_internado", "sum"),
224             media_permanencia_dias=(
225                 "tempo_de_permanencia_dias",
226                 "mean",
227             ),
228             media_leitos_disponiveis=(
229                 "numero_de_leitos_disponiveis",
230                 "mean",
231             ),
232             media_taxa_ocupacao=(
233                 "taxa_ocupacao_avc_itu_pneumonia",
234                 "mean",
235             ),
236             custo_estimado_total=("custo_estimado", "sum"),
237         )
238     )
239
240     # Arredondamento apenas para apresentação/armazenamento analítico.
241     indicadores["media_permanencia_dias"] = (
242         indicadores["media_permanencia_dias"].round(2)
243     )
244     indicadores["media_leitos_disponiveis"] = (
245         indicadores["media_leitos_disponiveis"].round(2)
246     )
247     indicadores["media_taxa_ocupacao"] = (
248         indicadores["media_taxa_ocupacao"].round(6)
249     )
250     indicadores["custo_estimado_total"] = (
251         indicadores["custo_estimado_total"].round(2)
252     )
253
254     indicadores.to_csv(PROCESSED_FILE, index=False)
255
256     LOGGER.info("Transformação concluída.")
257     LOGGER.info(
258         "Linhas de origem processadas: %s",
259         len(df),
260     )
```

Anexo A - continuação

```

261     LOGGER.info(
262         "Registros analíticos gerados: %s",
263         len(indicadores),
264     )
265     LOGGER.info(
266         "Granularidade: hospital + competência + diagnóstico"
267     )
268     LOGGER.info("Arquivo processado: %s", PROCESSED_FILE)
269     LOGGER.info("TRANSFORMAÇÃO: OK")
270
271     return {
272         "processed_path": str(PROCESSED_FILE),
273         "linhas_origem": int(info_ingestao["linhas_origem"]),
274         "registros_analiticos": int(len(indicadores)),
275     }
276
277 @task(task_id="validacao_dados")
278 def validacao_dados(info_transformacao: dict) -> dict:
279     """
280     Camada de qualidade:
281     impede a carga se as regras essenciais de integridade falharem.
282     """
283     LOGGER.info("===== DATACROSS | VALIDAÇÃO =====")
284
285     df = pd.read_csv(
286         info_transformacao["processed_path"],
287         parse_dates=["competencia"],
288     )
289
290     expected_total = int(info_transformacao["linhas_origem"])
291     aggregated_total = int(df["total_internacoes"].sum())
292
293     checks = {
294         "dataset_nao_vazio": len(df) > 0,
295         "total_origem_preservado": aggregated_total == expected_total,
296         "chaves_sem_nulos": not df[
297             [
298                 "id_do_hospital",
299                 "nome_do_hospital",
300                 "competencia",
301                 "doenca_detectada",
302             ]
303             .isna().any().any(),
304         "internados_consistentes": bool(
305             (df["total_internados"] <= df["total_internacoes"]).all()
306         ),
307         "permanencia_nao_negativa": bool(
308             (df["media_permanencia_dias"] >= 0).all()
309         ),
310         "ocupacao_intervalo_valido": bool(
311             df["media_taxa_ocupacao"].between(0, 1).all()
312         ),
313         "sem_duplicidade_granularidade": not df.duplicated(
314             subset=[
315                 "id_do_hospital",
316                 "competencia",
317                 "doenca_detectada",
318             ]
319             ).any(),
320     }
321
322     for check_name, result in checks.items():
323         LOGGER.info(
324             "[VALIDAÇÃO] %-35s : %s",
325             check_name,

```

Anexo A - continuação

```

326         "OK" if result else "FALHA",
327     )
328
329     LOGGER.info(
330         "[VALIDAÇÃO] Linhas na origem: %s",
331         expected_total,
332     )
333     LOGGER.info(
334         "[VALIDAÇÃO] Soma das interações agregadas: %s",
335         aggregated_total,
336     )
337     LOGGER.info(
338         "[VALIDAÇÃO] Registros analíticos: %s",
339         len(df),
340     )
341
342     failed = [name for name, result in checks.items() if not result]
343     if failed:
344         raise ValueError(
345             "Pipeline reprovado nas validações: " + " ", ".join(failed)
346         )
347
348     validation_payload = {
349         "status": "APROVADO",
350         "linhas_origem": expected_total,
351         "internacoes_agregadas": aggregated_total,
352         "registros_analiticos": int(len(df)),
353         "checks": checks,
354     }
355
356     with open(VALIDATION_FILE, "w", encoding="utf-8") as file:
357         json.dump(
358             validation_payload,
359             file,
360             ensure_ascii=False,
361             indent=2,
362         )
363
364     LOGGER.info("[VALIDAÇÃO] Pipeline aprovado para carga analítica.")
365     LOGGER.info("VALIDAÇÃO: OK")
366
367     return {
368         "processed_path": info_transformacao["processed_path"],
369         "validation_path": str(VALIDATION_FILE),
370         "linhas_origem": expected_total,
371         "registros_analiticos": int(len(df)),
372     }
373
374 @task(task_id="carga_analitica_oracle")
375 def carga_analitica_oracle(info_validacao: dict) -> dict:
376     """
377     Serving Layer:
378     executa carga full-refresh do data mart no Oracle Autonomous 26 AI.
379     """
380     LOGGER.info("===== DATACROSS | CARGA ANALÍTICA =====")
381
382     df = pd.read_csv(
383         info_validacao["processed_path"],
384         parse_dates=["competencia"],
385     )
386
387     conn = _oracle_connection()
388     cur = conn.cursor()
389
390     create_table_sql = f"""

```

Anexo A - continuação

```

391 CREATE TABLE {ORACLE_TABLE} (
392     ID_HOSPITAL NUMBER(10) NOT NULL,
393     NOME_HOSPITAL VARCHAR2(120) NOT NULL,
394     COMPETENCIA DATE NOT NULL,
395     DOENCA_DETECTADA VARCHAR2(100) NOT NULL,
396     TOTAL_INTERNAÇÕES NUMBER(10) NOT NULL,
397     TOTAL_INTERNADOS NUMBER(10) NOT NULL,
398     MEDIA_PERMANENCIA_DIAS NUMBER(10,2),
399     MEDIA_LEITOS_DISPONIVEIS NUMBER(10,2),
400     MEDIA_TAXA_OCUPACAO NUMBER(10,6),
401     CUSTO_ESTIMADO_TOTAL NUMBER(16,2),
402     DT_CARGA TIMESTAMP DEFAULT SYSTIMESTAMP,
403     CONSTRAINT PK_T_INDICADORES_AIRFLOW
404         PRIMARY KEY (
405             ID_HOSPITAL,
406             COMPETENCIA,
407             DOENCA_DETECTADA
408         )
409 )
410 """
411
412 try:
413     cur.execute(create_table_sql)
414     LOGGER.info("Tabela %s criada no Oracle.", ORACLE_TABLE)
415 except oracledb.DatabaseError as exc:
416     error_obj = exc.args[0]
417     if getattr(error_obj, "code", None) == 955:
418         LOGGER.info(
419             "Tabela %s já existe; será reutilizada.",
420             ORACLE_TABLE,
421         )
422     else:
423         cur.close()
424         conn.close()
425         raise
426
427 # Full refresh controlado para tornar a DAG idempotente.
428 cur.execute(f"DELETE FROM {ORACLE_TABLE}")
429
430 insert_sql = f"""
431 INSERT INTO {ORACLE_TABLE} (
432     ID_HOSPITAL,
433     NOME_HOSPITAL,
434     COMPETENCIA,
435     DOENCA_DETECTADA,
436     TOTAL_INTERNAÇÕES,
437     TOTAL_INTERNADOS,
438     MEDIA_PERMANENCIA_DIAS,
439     MEDIA_LEITOS_DISPONIVEIS,
440     MEDIA_TAXA_OCUPACAO,
441     CUSTO_ESTIMADO_TOTAL
442 )
443 VALUES (
444     :1, :2, :3, :4, :5,
445     :6, :7, :8, :9, :10
446 )
447 """
448
449 rows = []
450 for row in df.itertuples(index=False):
451     rows.append(
452         (
453             int(row.id_do_hospital),
454             str(row.nome_do_hospital),
455             row.competencia.to_pydatetime(),

```

Anexo A - continuação

```

456         str(row.doenca_detectada),
457         int(row.total_internacoes),
458         int(row.total_internados),
459         float(row.media_permanencia_dias),
460         float(row.media_leitos_disponiveis),
461         float(row.media_taxa_ocupacao),
462         float(row.custo_estimado_total),
463     )
464 )
465
466 cur.executemany(insert_sql, rows)
467 conn.commit()
468
469 LOGGER.info(
470     "Registros carregados em %s: %s",
471     ORACLE_TABLE,
472     len(rows),
473 )
474 LOGGER.info("Estratégia de carga: FULL REFRESH idempotente")
475 LOGGER.info("CARGA ANALÍTICA ORACLE: OK")
476
477 cur.close()
478 conn.close()
479
480 return {
481     "tabela": ORACLE_TABLE,
482     "registros_carregados": int(len(rows)),
483     "linhas_origem": int(info_validacao["linhas_origem"]),
484 }
485
486 @task(task_id="validacao_carga_oracle")
487 def validacao_carga_oracle(info_carga: dict) -> dict:
488     """
489     Validação pós-carga:
490     compara Oracle com os totais aprovados antes da carga.
491     """
492     LOGGER.info(
493         "===== DATACROSS | VALIDAÇÃO PÓS-CARGA ====="
494     )
495
496     conn = _oracle_connection()
497     cur = conn.cursor()
498
499     cur.execute(
500         f"""
501         SELECT
502             COUNT(*) AS REGISTROS_ANALITICOS,
503             SUM(TOTAL_INTERNAcoes) AS TOTAL_INTERNAcoes,
504             SUM(TOTAL_INTERNADOS) AS TOTAL_INTERNADOS
505         FROM {ORACLE_TABLE}
506         """
507     )
508
509     registros, total_internacoes, total_internados = cur.fetchone()
510
511     cur.close()
512     conn.close()
513
514     registros = int(registros or 0)
515     total_internacoes = int(total_internacoes or 0)
516     total_internados = int(total_internados or 0)
517
518     if registros != int(info_carga["registros_carregados"]):
519         raise ValueError(
520             "Quantidade de registros no Oracle difere da carga realizada."

```

Anexo A - continuação

```
521         )
522
523     if total_internacoes != int(info_carga["linhas_origem"]):
524         raise ValueError(
525             "Total de internações no Oracle não corresponde à origem."
526         )
527
528     LOGGER.info(
529         "[PÓS-CARGA] Registros analíticos Oracle: %s",
530         registros,
531     )
532     LOGGER.info(
533         "[PÓS-CARGA] Soma TOTAL_INTERNACOES: %s",
534         total_internacoes,
535     )
536     LOGGER.info(
537         "[PÓS-CARGA] Soma TOTAL_INTERNADOS: %s",
538         total_internados,
539     )
540     LOGGER.info("[PÓS-CARGA] Integridade entre origem e Oracle: OK")
541     LOGGER.info("PIPELINE DATACROSS CONCLUÍDO COM SUCESSO.")
542
543     return {
544         "status": "SUCESSO",
545         "tabela": ORACLE_TABLE,
546         "registros_analiticos": registros,
547         "total_internacoes": total_internacoes,
548         "total_internados": total_internados,
549     }
550
551 ingestao = ingestao_dataset()
552 transformacao = transformacao_dados(ingestao)
553 validacao = validacao_dados(transformacao)
554 carga = carga_analitica_oracle(validacao)
555 validacao_carga_oracle(carga)
556
557
558 datacross_pipeline()
```